

UNIVERSIDADE DE BRASÍLIA
INSTITUTO DE CIÊNCIAS EXATAS
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO

**CIC0099 ORGANIZAÇÃO E ARQUITETURA DE
COMPUTADORES**

Trabalho I: Memória do RISCv

OBJETIVO

Este trabalho consiste na simulação de instruções de acesso à memória do RISCv RV32I na linguagem C.

DESCRIÇÃO

Tipos de dados

C: Utilizar os tipos de dados definidos em `<stdint.h>`:

uint32_t, uint8_t : inteiro sem sinal de 32 e 8 bits, respectivamente.

int32_t, int8_t : inteiro com sinal de 32 e 8 bits, respectivamente.

Memória

A memória é simulada como um vetor de *bytes*. Cada índice do vetor é um endereço de memória.

C:

```
#define MEM_SIZE 16384
int8_t mem[MEM_SIZE];
```

Ou seja, a memória é um arranjo de 16KBytes.

Funções de acesso à Memória

As funções a serem implementadas são:

- *lb(reg, kte)*: lê um byte da memória e o converte para um inteiro de 32 bits estendendo o sinal do byte. Retorna o inteiro de 32 bits.
- *lbu(reg, kte)*: lê um byte da memória e o converte para um inteiro de 32 bits sem sinal (valor positivo). Retorna o inteiro de 32 bits.
- *lw(reg, kte)*: lê uma palavra de 32 bits da memória e retorna o seu valor.
- *sb(reg, kte, byte)*: escreve o byte passado como parâmetro na memória.
- *sw(reg, kte, word)*: escreve os 4 bytes de word na memória, colocando o menos significativo no endereço especificado e os outros nos endereços de *byte* seguintes.

Endereçamento da memória: todas as funções recebem dois parâmetros, *reg* e *kte*. O endereço de byte é dado pela soma dos dois:

$$address = reg + kte$$

No caso das funções *lw* e *sw*, os endereços devem ser múltiplos de 4.

Sugestões: é possível utilizar operações de mascaramento e deslocamento para processar as palavras.

Ex: extração dos bytes de uma palavra de 32 bits *word*:

seja *word* = 0x01020304 temos que:

byte0 = *word* & 0xFF retorna o primeiro byte, 0x04.

byte1 = (*word* >> 8) & 0xFF produz o segundo byte de *word*, 0x03.

byte2 = (*word* >> 16) & 0xFF produz o terceiro byte de *word*, 0x02.

byte3 = (*word* >> 24) & 0xFF produz o quarto byte de *word*, 0x01.

Ex: construção de uma palavra de 32 bits a partir de 4 bytes usando a operação lógica OU ("|" em C):

word = (*byte3* << 24) | (*byte2* << 16) | (*byte1* << 8) | *byte0*;

Por exemplo, a sequência de código abaixo:

```
sb(7, 0, 0xaa);  
word = lb(7, 0);  
printf("word_10 = %d\n", word);  
printf("word_x = %08x\n", hex(word));
```

imprime na tela:

```
word_10 = -86  
word_x = 0xffffffffaa
```

`printf` é uma função C (não é comando da linguagem) que imprime dados na console usando um *string* de formatação.

Valores são inseridos no string utilizando sequências que iniciam com "%". Assim, "%d" indica valor decimal com sinal e "%08x" indica um número com 8 dígitos hexadecimais.

Notem a diferença entre `lbu()` e `lb()`:

```
word = lbu(7, 0);  
printf("word_10 = %d\n", word);  
printf("word_x = %08x\n", hex(word));
```

imprime na tela:

```
word_10 = 170  
word_x = 0x000000aa
```

Verificação

As funções devem ser inseridas no questionário disponível na página do Aprender da disciplina. Recomenda-se desenvolver e testar em uma IDE as funções antes de submeter ao CodeRunner.

Teste da função `lb()`:

O teste da função `lb()` escreve um conteúdo na memória e depois verifica se `lb()` o lê corretamente, considerando a extensão correta do sinal. No teste abaixo, o valor é impresso em hexadecimal.

```
void teste_lb(void)
{
    mem[100] = (int8_t)0xEF;
    mem[101] = (int8_t)0xAD;
    mem[102] = (int8_t)0x7C;
    mem[103] = (int8_t)0x7B;

    printf("lb(100, 0) = 0x%08x\n", (uint32_t)lb(100, 0));
    printf("lb(100, 1) = 0x%08x\n", (uint32_t)lb(100, 1));
    printf("lb(100, 2) = 0x%08x\n", (uint32_t)lb(100, 2));
    printf("lb(100, 3) = 0x%08x\n", (uint32_t)lb(100, 3));
}
```

Resposta esperada:

```
lb(100, 0) = 0xffffffffef    // extensão de sinal negativo
lb(100, 1) = 0xffffffffad    // extensão de sinal negativo
lb(100, 2) = 0x0000007c      // extensão de sinal positivo
lb(100, 3) = 0x0000007b      // extensão de sinal positivo
```